

CURSO: Desenvolvimento de Sistemas		UC Teste de Sistemas	
INSTRUTOR:	Pietra Medeiros		
NOME DO ALUNO:	Vanderson da Silva		TURMA: HTC-DDS-19

ATIVIDADE AVALIATIVA

- 1) Explique a importância do alinhamento entre o modelo de desenvolvimento de software e a definição dos níveis de teste. Como esse alinhamento contribui para a qualidade do produto final?

Esse alinhamento garante a qualidade do software. Ao definir os níveis de teste de acordo com o modelo de desenvolvimento utilizado, a equipe consegue identificar e corrigir erros e falhas de forma mais eficiente e precoce, reduzindo riscos e custos.

- 2) Compare pelo menos dois modelos de desenvolvimento de software (por exemplo, Cascata e Incremental) e descreva como a atividade de testes se adapta em cada um deles.

No modelo incremental, o desenvolvimento é feito em partes menores e entregue gradualmente. No Incremental a atividade de testes visa verificar a funcionalidade e a qualidade de cada incremento do sistema, permitindo que os erros sejam corrigidos rapidamente e que o feedback do cliente seja incorporado no desenvolvimento. Requer uma comunicação constante entre a equipe e o cliente. Agrega mais flexibilidade para adaptações e alterações, pois os incrementos podem ser ajustados com base no feedback do cliente.

Na modelo cascata, o desenvolvimento é feito de forma sequencial e linear, onde cada fase do desenvolvimento (requisitos, design, implementação, testes, etc.) é concluída antes de iniciar a seguinte. Testes são realizados principalmente no final do ciclo de desenvolvimento, após a implementação de todas as funcionalidades, visam verificar a integridade e a qualidade do sistema como um todo, antes de sua entrega ao cliente. Devido à natureza sequencial do modelo, os testes podem ser mais concentrados e demorar mais tempo para serem concluídos. Facilita o planejamento e controle, pois as etapas são bem definidas. Possui pouca flexibilidade para adaptações e alterações, pois mudanças em uma fase tardia podem afetar todo o projeto.

- 3) Quais são os principais objetivos ao definir uma estratégia de teste? Como ela influencia a eficácia da equipe de testes?

Os principais objetivos são identificar e mitigar riscos, garantir a qualidade do produto final, otimizar os recursos e a eficiência da equipe de testes e facilitar a comunicação entre as partes interessadas. A estratégia de testes influencia a eficácia da equipe ao definir um plano de ação claro, fornecendo uma abordagem estruturada e consistente para as atividades de teste, o que leva a melhor qualidade e confiabilidade dos produtos de software.

- 4) Descreva com suas palavras o que deve ser levado em consideração ao definir “o que testar” em um sistema. Dê exemplos de critérios que podem ser utilizados.

Ao definir "o que testar" em um sistema, é fundamental considerar os requisitos, o escopo e o contexto do sistema, bem como a experiência do usuário e as possíveis falhas. Isso envolve identificar pontos críticos, áreas de risco e funcionalidades que mais afetam a qualidade e o desempenho do sistema.

Funcionalidade, segurança, desempenho, usabilidade e compatibilidade são os critérios que devem ser utilizados para definir a abrangência dos testes.

Considerações:

Requisitos e Especificações: A análise dos requisitos do sistema, incluindo funcionalidades, desempenho e segurança, é fundamental. Os testes devem verificar se o sistema atende a esses requisitos.

Funções Essenciais: Identificar as funcionalidades mais críticas e de maior impacto no usuário, como login, navegação, e transações financeiras. Estas devem ser testadas de forma mais rigorosa.

Áreas de Risco: Avaliar as áreas do sistema que são mais propensas a erros ou falhas, como cálculos complexos, integração com sistemas externos ou funcionalidades que tratam dados sensíveis. Estes devem ser testados de forma mais aprofundada.

Experiência do Usuário (UX): Verificar a usabilidade do sistema, incluindo a facilidade de uso, a clareza das informações, e a adequação da interface gráfica. Testes de UX podem ajudar a identificar problemas de design que afetam a experiência do usuário.

Contexto de Uso: Considere o ambiente em que o sistema será utilizado (ambiente de produção, testes, etc.) e a sua interdependência com outros sistemas, infraestrutura e usuários.

Performance: Teste o sistema sob diferentes condições de carga (número de usuários, tráfego) para garantir a sua capacidade de responder a diferentes demandas.

Segurança: Avalie possíveis vulnerabilidades e ataques, como ataques de força bruta, injeção de código, ou acesso não autorizado.

Crítérios:

Função: O sistema deve permitir que um usuário cadastre um novo produto, incluindo o nome, descrição, preço e categoria.

Segurança: O sistema deve garantir que apenas usuários autenticados possam acessar informações confidenciais.

Desempenho: O sistema deve carregar páginas rapidamente, mesmo com grande volume de dados e muitos usuários conectados simultaneamente.

Usabilidade: A interface do sistema deve ser intuitiva, fácil de usar e visualmente atraente, permitindo que o usuário execute tarefas com facilidade e rapidez.

Compatibilidade: Verificar se é compatível com diferentes navegadores, sistemas operacionais, dispositivos e versões de software.

- 5) Explique o conceito de 'níveis de teste' e descreva brevemente cada um deles: teste de unidade, teste de integração, teste de sistema e teste de aceitação.

Os "níveis de teste" referem-se a diferentes abordagens para testar um sistema, desde as menores unidades de código até o sistema completo. Os principais níveis são: teste de unidade, teste de integração, teste de sistema e teste de aceitação.

Testes de Unidade: Testam individualmente as menores partes do código, como funções ou métodos, para verificar se funcionam corretamente de forma isolada, antes de serem integradas no sistema.

Testes de Integração: Verificam a interação entre diferentes componentes ou módulos do sistema, garantindo que eles funcionem bem juntos e que a comunicação entre eles seja eficaz.

Testes de Sistema: Testam o sistema completo como um todo, verificando se ele atende aos requisitos funcionais e não funcionais, e se a interação entre os componentes é adequada.

Testes de Aceitação: Testam o sistema com base nas necessidades e expectativas dos usuários finais, para verificar se ele atende às suas necessidades e se a solução entregue é satisfatória.

- 6) Qual o papel da documentação (como logs, relatórios de defeitos e relatórios gerenciais) no ciclo de vida dos testes? Por que é importante manter esse registro?

Tem um papel crucial na gestão de qualidade e na tomada de decisões informadas ao longo do ciclo de vida dos testes. Manter esses registros é importante porque facilita a reprodução, a análise e a correção de problemas, além de fornecer métricas para avaliar o progresso dos testes e a eficácia do processo de desenvolvimento.

A documentação detalhada facilita a identificação da causa raiz dos defeitos, a reprodução dos problemas e a implementação de soluções eficazes.

Ao analisar os resultados dos testes e os relatórios gerenciais, é possível identificar áreas de melhoria no processo de desenvolvimento e implementar ações corretivas para evitar que os problemas se repitam.

Os relatórios gerenciais fornecem informações cruciais para a gestão, permitindo que as decisões sejam tomadas com base em dados concretos e evitando a tomada de decisões equivocadas.

A documentação permite rastrear a evolução de um defeito desde sua identificação até sua correção, garantindo que o problema seja resolvido e que a qualidade do software seja mantida.

A documentação facilita a comunicação entre as diferentes equipes envolvidas no projeto, permitindo que todos estejam alinhados com os objetivos e o progresso do projeto.

- 7) Reflita sobre o papel das reuniões de lições aprendidas após a execução de testes. Quais benefícios essa prática pode trazer em projetos futuros?

São essenciais para a melhoria contínua em projetos, incluindo aqueles com execução de testes. Elas permitem identificar erros e acertos, promovendo a cultura de aprendizado e aprimoramento de processos em projetos futuros. Ao analisar o que funcionou bem e o que poderia ser melhorado, as equipes podem evitar repetições de problemas, otimizar recursos e aumentar a eficiência.

Principais benefícios:

Melhoria na tomada de decisões: Com base nas lições aprendidas, os profissionais podem tomar decisões e podem evitar erros passados e otimizar processos.

Redução de riscos: A visão de riscos e problemas em projetos anteriores permite a adoção de medidas preventivas em projetos futuros.

Otimização de recursos: Conhecendo as áreas de ineficiência, as equipes podem reduzir custos e otimizar o uso de recursos em projetos futuros.

Fortalecimento da equipe: A identificação de acertos e erros em projetos anteriores permite que a equipe fortaleça seus pontos fortes e trabalhe em seus pontos fracos, aumentando a confiança e a produtividade.

Evita a reincidência de erros: As lições aprendidas ajudam a evitar que erros sejam repetidos em projetos futuros.

Documentação de boas práticas:

As lições aprendidas podem ser documentadas e compartilhadas com outras equipes, promovendo a disseminação de boas práticas e aprimorando o conhecimento organizacional.

Aumento da satisfação do cliente: Evitando erros se promove a qualidade dos produtos e serviços, aumentando a satisfação do cliente.

Aprimoramento do processo de teste: A análise das lições aprendidas, permite identificar áreas de melhoria no processo de teste, como aprimorar a cobertura de testes, aprimorar a criação de casos de teste e aprimorar a análise de resultados de teste.

- 8) Quais são os riscos de não realizar um planejamento adequado da estratégia e do escopo de testes em um projeto de software? Dê exemplos reais ou hipotéticos.

Baixa produtividade, retrabalho, custo elevado podendo, pode ser a causa para que um projeto seja considerado inviável e ou cancelado.

A falta de um planejamento adequado ou desorganizado impacta diretamente a produtividade das equipes. Sem um planejamento eficiente ou mal documentado, colaboradores passam mais tempo procurando documentos do que realmente executando suas tarefas, o que gera atrasos e compromete o fluxo de trabalho e aumentam os custos.

- 9) A partir dos conteúdos dos slides, explique como a avaliação dos resultados dos testes pode impactar a tomada de decisões em fases posteriores do projeto.

Estes resultados podem impactar na tomada de decisão quanto o direcionamento ou redirecionamento de recursos, na correção e implementação de processos produtivos, reduzir o custo estimado em um projeto bem como evitar o atraso nas entregas.

- 10) Imagine que você é o responsável pelos testes em um novo projeto. Descreva um plano básico, incluindo o escopo, as estratégias, os níveis de teste e como você avaliaria os resultados.

- Escopo dos Testes -

O escopo define o que será e o que não será testado. Neste projeto, o escopo inclui:

Funcionalidades principais do sistema (ex: login, cadastro, processamento de pedidos).

Integrações com APIs externas "se houverem".

Interface do usuário em dispositivos desktop e mobile.

Regras de negócio críticas.

- Estratégias de Teste -

A estratégia define a abordagem geral que será usada para garantir a qualidade.

Testes Automatizados para cenários repetitivos, como testes de regressão e testes de API "se houver acesso a alguma".

Testes Manuais para novas funcionalidades, exploração de interface e testes de usabilidade.

TDD (Test-Driven Development) quando aplicável, para componentes críticos.

CI/CD com execução de testes automatizados a cada push para branches principais.

- Níveis de Teste -

Teste Unitário: Desenvolvedores criam testes para funções e métodos individuais. Frameworks como JUnit, pytest ou Jest, dependendo da linguagem.

Teste de Integração: Verifica a interação entre módulos (ex: backend e banco de dados, ou frontend e API). Foco em falhas de comunicação.

Teste de Sistema: Avalia o comportamento do sistema como um todo. Testes de ponta a ponta (E2E).

Teste de Aceitação: Realizados com base nos critérios definidos com o cliente ou PO.

- Avaliação dos Resultados (Critérios) -

Taxa de Passes/Falhas: Percentual de casos que passaram vs. falharam.

Cobertura de Testes: Percentual de código coberto pelos testes automatizados.

Métricas de Defeitos: Número, gravidade e tempo médio de correção dos bugs encontrados.

Feedback do Usuário (caso aplicável): Para validar usabilidade e aderência às expectativas.

Após cada ciclo de testes (ex: Sprint), os resultados são documentados e compartilhados com a equipe. Análises retroativas (retrospectivas) são realizadas para melhorar continuamente o processo de testes.